



UNIVERSIDAD DE COLIMA

Facultad de Ingeniería Mecánica y Eléctrica



Práctica de Laboratorio de
Programación Funcional
Creando un Chatbot ELIZA con Python

Dr. Walter Alexander Mata López

Coquimatlán, Colima, septiembre de 2024





Contenido

Práctica de Laboratorio: Creando un Chatbot ELIZA con Python.....	3
Introducción	3
¿Qué es ELIZA?	3
Ejemplo de una Conversación con ELIZA	3
Desarrollo de la Aplicación ELIZA Paso a Paso.....	4
Paso 1: El “Cerebro” de ELIZA - Las Reglas de Conversación.....	4
Código 1: El “Cerebro” de ELIZA - Las Reglas de Conversación	4
Explicación del Código 1:	5
¿Qué es la tupla de la clave?	5
¿Qué es la lista del valor?	5
Ventajas de este diseño	5
Respuestas genéricas en ELIZA	6
Qué es exactamente.....	6
Mantenimiento y ampliación	6
Resumen del Código 1:	7
Paso 2: El Reflejo Terapéutico - Cambiando Puntos de Vista	7
Código 2: El Reflejo Terapéutico - Cambiando Puntos de Vista	7
Explicación del Código 2:	7
Paso 3: El Corazón del Chatbot - La Función para Responder	8
Código 3: El Corazón del Chatbot - La Función para Responder	8
Explicación del Código 3:	9
Paso 4: La Conversación - El Bucle Principal.....	10
Código 4: La Conversación - El Bucle Principal	10
Explicación del Código 4:	11
Implementación con Streamlit	12
Paso 1: Instalar Streamlit.....	12
Explicación del Código 5:	12
Paso 2: Crear el Archivo de la Aplicación Web	12
Explicación del Código 6:	14
Paso 3: Ejecutar la Aplicación	15
Explicación del Código 7:	15
Conclusión	16



Práctica de Laboratorio: Creando un Chatbot ELIZA con Python

Introducción

En esta guía veremos cómo construir tu propio chatbot inspirado en **ELIZA**, uno de los primeros programas de inteligencia artificial conversacional. El objetivo es que **aprendas los fundamentos de Python directamente en la acción**. Cada concepto se introducirá justo cuando lo necesitemos para construir nuestro programa, haciendo el aprendizaje más intuitivo y aplicado.

No usaremos código ni librerías complejas, construiremos toda la lógica con funciones básicas de Python, demostrando que se pueden lograr resultados sorprendentes con herramientas sencillas. Al final, convertiremos nuestro chatbot en una aplicación web interactiva con Streamlit.

¿Qué es ELIZA?

ELIZA es un programa informático creado entre 1964 y 1966 en el MIT por Joseph Weizenbaum. Su propósito era simular a una psicoterapeuta rogeriana, un tipo de terapia que se basa en reflejar las propias palabras del paciente para animarlo a seguir hablando. ELIZA no “entendía” la conversación, sino que usaba una ingeniosa técnica de **búsqueda de palabras clave y sustitución** para dar la ilusión de comprensión. Este fenómeno, donde las personas atribuyen inteligencia y emociones a un programa simple, se conoce como el “efecto ELIZA”.

Ejemplo de una Conversación con ELIZA

- **ELIZA:** Hola. Soy Eliza. Por favor, dime tu problema.
- **Usuario:** Mi jefe me tiene estresado.
- **ELIZA:** ¿Tu jefe te tiene estresado? Cuéntame más.
- **Usuario:** Siento que no valora mi trabajo.
- **ELIZA:** Lamento oír que te sientes así. ¿Por qué crees que no valora tu trabajo?
- **Usuario:** Porque nunca me da las gracias.
- **ELIZA:** Ya veo. Por favor, continúa.





Desarrollo de la Aplicación ELIZA Paso a Paso

Comencemos a construir nuestro chatbot. Cada paso introducirá un nuevo bloque de código y explicará los conceptos de Python que estamos utilizando.

Paso 1: El “Cerebro” de ELIZA - Las Reglas de Conversación

Lo primero que necesita nuestro chatbot es un conjunto de reglas. Usaremos una de las estructuras de datos más versátiles de Python: el **diccionario (dict)**. Un diccionario nos permite almacenar información en pares de clave: valor, como un diccionario real. En nuestro caso, las claves serán las “palabras clave” que buscaremos, y los valores serán las posibles respuestas.

Código 1: El “Cerebro” de ELIZA - Las Reglas de Conversación

```
import random

respuestas_por_palabra_clave = {
    ("hola", "buenas"):
        ["Hola. Soy Eliza. Por favor, dime tu problema.", "Hola, ¿qué te trae por aquí?"],
    ("adiós", "hasta luego"):
        ["Adiós. Gracias por hablar conmigo.", "Hasta luego, que tengas un buen día."],
    ("nombre",):
        ["No necesito saber tu nombre, pero gracias por compartirlo.", "Un nombre no define quién eres, pero es un placer."],
    ("necesito", "quiero"):
        ["¿Por qué lo necesitas?", "¿Qué pasaría si no lo tuvieras?", "Entiendo que lo deseas."],
    ("siento", "estoy"):
        ["¿Por qué te sientes así?", "Lamento oír que te sientes así. Cuéntame más.", "Describe con más detalle ese sentimiento."],
    ("recuerdo",):
        ["¿Qué más recuerdas sobre eso?", "¿Cómo te hace sentir ese recuerdo?", "Los recuerdos son la clave del pasado."],
    ("madre", "mamá", "padre", "papá", "hermano", "hermana"):
        ["Cuéntame más sobre tu familia.", "La familia es un tema importante. ¿Cómo te llevas con ellos?"],
    ("eres", "sos"):
        ["¿Qué te hace pensar eso de mí?", "¿Realmente importa lo que soy? Hablemos de ti."],
}

respuestas_genericas = [
```





```
"Ya veo.",  
"Por favor, continúa.",  
"¿Puedes darme más detalles?",  
"¿En qué más piensas?",  
"Entiendo.",  
"Continúa, te escucho."
```

Explicación del Código 1:

Como puedes apreciar, en ese diccionario las **claves son tuplas** y los **valores son listas**. Veamos la explicación precisa de cada una y por qué se usan así.

¿Qué es la **tupla** de la clave?

Ejemplos: ("hola", "buenas"), ("nombre",), ("madre", "mamá", "padre", ...)

- **Agrupar sinónimos o palabras clave equivalentes** que compartirán **el mismo conjunto de respuestas**.
- **Es inmutable** (no se puede cambiar después de creada) y, por eso, **es hashable**: puede usarse como clave de un diccionario. (Una lista no podría ser clave).
- Permite preguntar **“¿la palabra de entrada está en esta tupla?”** para decidir si ese grupo aplica.
- **Detalle importante del caso unitario**: ("nombre",) lleva **coma final**. Sin esa coma, Python lo interpretaría como una cadena entre paréntesis y **no** como tupla.

¿Qué es la **lista** del valor?

Ejemplos:

```
["Hola. Soy Eliza. Por favor, dime tu problema.", "Hola, ¿qué te trae por aquí?"]  
["¿Por qué lo necesitas?", "¿Qué pasaría si no lo tuvieras?", "Entiendo que lo desees."]
```

- **Colección ordenada y mutable** de textos de respuesta **alternativos** para una misma intención.
- Se usa con `random.choice(lista)` para **variar** la salida y evitar respuestas repetitivas.
- Al ser **mutable**, puedes **añadir, quitar o reordenar** respuestas sin tocar la estructura general:

Ventajas de este diseño

- **Evita duplicación**: un solo grupo (tupla) cubre varios sinónimos.
- **Clave segura**: la tupla es **hashable**; una lista no serviría como clave.





- **Respuestas variadas:** la lista permite **rotar** mensajes naturalmente.

Con esto queda clara la función de la **tupla (clave, inmutable, agrupa sinónimos)** y de la **lista (valor, respuestas alternativas y mutables)** en tu “cerebro” de ELIZA.

Respuestas genéricas en ELIZA

La lista `respuestas_genericas` funciona como un conjunto de **respuestas genéricas o de acompañamiento** (“backchannels”) que ELIZA puede usar cuando:

1. **No detecta ninguna palabra clave** en el mensaje del usuario (caso “sin intención”).
2. **Quiere mantener el flujo** mientras busca más información antes de formular algo más específico.
3. **Desea variar** las salidas para no sonar repetitiva aun cuando sí hay reglas, pero conviene no “quemarlas” en cada turno.

Qué es exactamente

`respuestas_genericas` es una **lista (list)**: una colección **ordenada y mutable** de cadenas.

- **Ordenada:** conserva el orden de inserción (por si decides recorrerla secuencialmente).
- **Mutable:** puedes añadir, quitar o reordenar respuestas en tiempo de ejecución.
- **Relleno significativo:** mantiene la conversación “viva” cuando no hay coincidencias en el diccionario de reglas.
- **Apoyo a la escucha activa:** frases breves que invitan a ampliar (coherentes con el estilo de ELIZA).
- **Variedad:** combinadas con `random.choice`, evitan respuestas idénticas en turnos consecutivos.

Mantenimiento y ampliación

- Añade variantes sinónimas (“Te escucho.”, “Cuéntame más.”, “¿Podrías ampliar un poco?”).
- Mantén el **tono consistente** (empático, breve, no directivo), acorde al enfoque clásico de ELIZA.
- Si localizas el bot a otras regiones, revisa **modismos** (“vale”, “continúa por favor”, etc.).

En otras palabras, `respuestas_genericas` es la **válvula de seguridad conversacional**: una lista mutable de frases breves que ELIZA usa para **sostener el diálogo, pedir**





ampliación y evitar silencios o repeticiones, especialmente cuando las reglas específicas no aplican.

Resumen del Código 1:

Como podrás ver, estamos creando el “cerebro” de nuestro chatbot usando un enfoque directo:

- **respuestas_por_palabra_clave:** Es un **diccionario (dict)**. Las claves de este diccionario son **tuplas (tuple)** que contienen las palabras clave que queremos detectar. Por ejemplo, la tupla ("hola", "buenas") agrupa sinónimos. El valor asociado a cada clave es una **lista (list)** de posibles respuestas. Usar una tupla como clave es una práctica común en Python cuando necesitas agrupar varios elementos inmutables.
- **respuestas_genericas:** Es una **lista (list)** de frases que el chatbot usará cuando no encuentre ninguna palabra clave en la entrada del usuario. Esto asegura que la conversación nunca se detenga.
- **import random:** Importamos la biblioteca random al principio, ya que la necesitaremos más adelante para elegir una respuesta al azar de nuestras listas.

Paso 2: El Reflejo Terapéutico - Cambiando Puntos de Vista

Una de las técnicas de ELIZA era tomar la frase del usuario y devolvérsela como una pregunta. Para ello, necesitaba cambiar los pronombres (por ejemplo, de “mi” a “tu”). Crearemos otro **diccionario** para manejar estas transformaciones.

Código 2: El Reflejo Terapéutico - Cambiando Puntos de Vista

```
cambio_pronombres = {  
    "yo": "tú",  
    "mi": "tu",  
    "mis": "tus",  
    "soy": "eres",  
    "estoy": "estás",  
    "me": "te"  
}
```

Explicación del Código 2:

Este es un truco simple pero muy efectivo. Creamos otro **diccionario (dict)** llamado cambio_pronombres.

- La **clave** es el pronombre o verbo en primera persona (ej. “yo”, “mi”, “soy”).
- El **valor** es su equivalente en segunda persona (ej. “tú”, “tu”, “eres”).





Más adelante, cuando nuestro chatbot detecte una frase que puede “reflejar”, recorrerá las palabras de la frase y usará este diccionario para hacer los cambios necesarios. Por ejemplo, si el usuario dice “*mi jefe me ignora*”, el chatbot podrá transformarlo en “*¿tu jefe te ignora?*”. Esto crea una sensación de que el chatbot está prestando atención y procesando la información, cuando en realidad solo está haciendo una sustitución de palabras.

Paso 3: El Corazón del Chatbot - La Función para Responder

Aquí es donde ocurre la magia. Crearemos una función que tome la entrada del usuario, la descomponga en palabras y decida cómo responder. Esta función será el núcleo de la lógica de nuestro programa.

Código 3: El Corazón del Chatbot - La Función para Responder

```
def generar_respuesta(entrada_usuario):
    # Convertimos la entrada a una lista de palabras en minúsculas.
    # Esto es un ejemplo de manipulación de cadenas (string) y creación de
    listas (list).
    palabras = entrada_usuario.lower().split()

    # Usamos un bucle 'for' para iterar sobre nuestro diccionario de reglas.
    for grupo_claves, respuestas in respuestas_por_palabra_clave.items():
        # Usamos otro bucle 'for' anidado para revisar cada palabra clave en
        el grupo.
        for clave in grupo_claves:
            # El operador 'in' es una forma simple y poderosa de verificar
            pertenencia.
            if clave in palabras:
                # Si encontramos una palabra clave, decidimos si reflejar o
                dar una respuesta estándar.
                # El operador '%' (módulo) nos ayuda a tomar una decisión
                simple.
                if clave in ["siento", "estoy", "necesito", "quiero"] and
                len(palabras) > 2:
                    # Reflejamos la frase del usuario, cambiando los
                    pronombres.
                    frase_reflejada = []
                    for palabra in palabras:
                        # Usamos .get() para buscar en el diccionario de
                        pronombres.
                        # Si la palabra no está, simplemente la usamos tal
                        cual.
                        frase_reflejada.append(cambio_pronombres.get(palabra,
                        palabra))
```





```
        # Unimos la lista de palabras para formar una nueva
        oración y la devolvemos.
        return "¿Dices que " + " ".join(frase_reflejada) + "?"
    else:
        # Si no, simplemente elegimos una respuesta al azar de la
        lista.

        return random.choice(respuestas)

# Si el bucle 'for' termina sin encontrar ninguna palabra clave,
devolvemos una respuesta genérica.
return random.choice(respuestas_genericas)
```

Explicación del Código 3:

Esta función es el núcleo de nuestro chatbot y aquí se aplican muchos de los conceptos de Python que mencionamos:

1. **def generar_respuesta(entrada_usuario)::** Define una **función** que toma la entrada del usuario como argumento.
2. **palabras = entrada_usuario.lower().split():** Aquí, tomamos la **cadena (str)** de entrada, la convertimos a minúsculas con **.lower()** y luego la dividimos en una **lista (list)** de palabras individuales con **.split()**. Este es un paso fundamental en casi cualquier tarea de procesamiento de lenguaje.
3. **for grupo_claves, respuestas in ...:** Usamos un **bucle for** para iterar sobre cada par (grupo de claves, lista de respuestas) en nuestro diccionario **respuestas_por_palabra_clave**.
4. **if clave in palabras::** Este es un **condicional if** que utiliza el operador **in** para comprobar si una de nuestras palabras clave existe en la lista de palabras del usuario. Es una forma muy eficiente de buscar elementos.
5. **Lógica de Reflejo:** Si la palabra clave es una que invita a la reflexión (como “siento” o “necesito”), construimos una respuesta reflejada. Creamos una nueva lista **frase_reflejada** y, mediante otro **bucle for**, la llenamos palabra por palabra, usando el diccionario **cambio_pronombres** para hacer las sustituciones. El método **.get(palabra, palabra)** es una forma segura de acceder a un diccionario: si la palabra no es una clave en el diccionario, simplemente devuelve la palabra original.
6. **return random.choice(respuestas):** Si no se cumple la condición de reflejo o si la palabra clave no lo requiere, simplemente usamos **random.choice()** para seleccionar y devolver una respuesta de la lista correspondiente.





7. **Respuesta Genérica:** Si el bucle principal termina y no se ha devuelto ninguna respuesta (lo que significa que no se encontró ninguna palabra clave), la función devuelve una respuesta de la lista `respuestas_genericas`.

Paso 4: La Conversación - El Bucle Principal

Finalmente, necesitamos un bucle que mantenga la conversación en marcha. Usaremos un bucle `while`, que se repetirá indefinidamente hasta que el usuario decida terminar la charla. Aquí es donde pondremos todo junto para crear la experiencia interactiva.

Código 4: La Conversación - El Bucle Principal

```
def eliza_chat():
    # Imprimimos un mensaje de bienvenida una sola vez.
    print("ELIZA: Hola. Soy Eliza. Por favor, dime tu problema. (Escribe 'adiós' para salir)")

    # Este es el bucle de control de flujo principal. Se ejecutará para siempre.
    while True:
        # input() es una función integrada de Python que detiene el programa y espera texto del usuario.
        entrada_usuario = input("TÚ: ")

        # Usamos un condicional 'if' para comprobar si el usuario quiere terminar.
        # El operador 'or' nos permite verificar múltiples condiciones.
        if entrada_usuario.lower() == "adiós" or entrada_usuario.lower() == "salir":
            print("ELIZA: Adiós. Gracias por hablar conmigo.")
            # La declaración 'break' rompe el bucle 'while' y termina el programa.
            break

        # Si no salimos, llamamos a nuestra función principal para obtener una respuesta.
        respuesta = generar_respuesta(entrada_usuario)
        # f-string (cadena formateada) es una forma moderna y legible de insertar variables en texto.
        print(f"ELIZA: {respuesta}")

# --- INICIO DE LA EJECUCIÓN ---
# Para probar el chatbot en tu libreta, simplemente descomenta y ejecuta la siguiente línea.
# eliza_chat()
```





Explicación del Código 4:

Este bloque une todas las piezas. Aquí aplicamos los conceptos de control de flujo:

1. **def eliza_chat()::** Define la función que encapsula toda la lógica de la conversación.
2. **while True::** Este es un **bucle infinito**. El código dentro de él se repetirá una y otra vez, permitiendo una conversación continua. Es el motor que mantiene a ELIZA “escuchando”.
3. **entrada_usuario = input("TÚ: ")**: La función `input()` pausa el programa y espera a que el usuario escriba algo y presione Enter. El texto ingresado se guarda en la **variable de tipo cadena (str)** `entrada_usuario`.
4. **if entrada_usuario.lower() == "adiós" ...**: Este es el **condicional if** que le da al usuario una salida. Compara la entrada del usuario (en minúsculas) con las palabras de salida. El uso del **operador lógico or** permite que cualquiera de las dos palabras funcione.
5. **break**: Si la condición del `if` es verdadera, la declaración `break` se ejecuta, rompiendo inmediatamente el bucle `while` y finalizando la conversación (y el programa).
6. **print(f"ELIZA: {respuesta}")**: Utilizamos una **f-string** para mostrar la respuesta del chatbot de una manera limpia y legible. Las f-strings son una característica muy útil de Python para formatear texto.

Al descomentar y ejecutar `eliza_chat()`, iniciarás la conversación en tu propia consola o libreta de Jupyter.





Implementación con Streamlit

Ahora que tenemos la lógica completa de nuestro chatbot, ¡vamos a darle una cara bonita! Usaremos **Streamlit** para convertir nuestro script de consola en una aplicación web interactiva.

Paso 1: Instalar Streamlit

Primero, necesitamos instalar la biblioteca. Puedes hacerlo directamente desde la libreta de Jupyter.

```
# Código 5: Instalación de Streamlit
!pip install streamlit
```

Explicación del Código 5:

Streamlit es una biblioteca de Python que te permite crear aplicaciones web para tus proyectos de datos y machine learning con muy poco esfuerzo. En lugar de tener que aprender sobre desarrollo web complejo (HTML, CSS, JavaScript), puedes usar comandos simples de Python para crear botones, cajas de texto, y mostrar resultados.

Este comando, para ser usado en una celda de Jupyter Notebook, utiliza **!pip** para instalar la biblioteca **streamlit** en tu entorno. Si estás trabajando en tu terminal local, simplemente ejecutarías `pip install streamlit`.

Paso 2: Crear el Archivo de la Aplicación Web

El siguiente código debe guardarse en un archivo separado llamado `app.py`. Este archivo contendrá toda la lógica de nuestro chatbot adaptada para funcionar con la interfaz de Streamlit.

```
# Código 6: El Archivo de la Aplicación Web (app.py)
import streamlit as st
import random

# --- Lógica del Chatbot (copiada de los pasos anteriores) ---

respuestas_por_palabra_clave = {
    ("hola", "buenas"):
        ["Hola. Soy Eliza. Por favor, dime tu problema.", "Hola, ¿qué te trae por aquí?"],
    ("adiós", "hasta luego"):
        ["Adiós. Gracias por hablar conmigo.", "Hasta luego, que tengas un buen día."],
    ("nombre",):
```





```
["No necesito saber tu nombre, pero gracias por compartirlo.", "Un
nombre no define quién eres, pero es un placer."],
("necesito", "quiero"):
    ["¿Por qué lo necesitas?", "¿Qué pasaría si no lo tuvieras?",
"Entiendo que lo deseas."],
("siento", "estoy"):
    ["¿Por qué te sientes así?", "Lamento oír que te sientes así. Cuéntame
más.", "Describe con más detalle ese sentimiento."],
("recuerdo",):
    ["¿Qué más recuerdas sobre eso?", "¿Cómo te hace sentir ese
recuerdo?", "Los recuerdos son la clave del pasado."],
("madre", "mamá", "padre", "papá", "hermano", "hermana"):
    ["Cuéntame más sobre tu familia.", "La familia es un tema importante.
¿Cómo te llevas con ellos?"],
("eres", "sos"):
    ["¿Qué te hace pensar eso de mí?", "¿Realmente importa lo que soy?
Hablemos de ti."],
}

respuestas_genericas = [
    "Ya veo.",
    "Por favor, continúa.",
    "¿Puedes darme más detalles?",
    "¿En qué más piensas?",
    "Entiendo.",
    "Continúa, te escucho."
]

cambio_pronombres = {
    "yo": "tú", "mi": "tu", "mis": "tus", "soy": "eres", "estoy": "estás",
    "me": "te"
}

def generar_respuesta(entrada_usuario):
    palabras = entrada_usuario.lower().split()
    for grupo_claves, respuestas in respuestas_por_palabra_clave.items():
        for clave in grupo_claves:
            if clave in palabras:
                if clave in ["siento", "estoy", "necesito", "quiero"] and
len(palabras) > 2:
                    frase_reflejada = []
                    for palabra in palabras:
                        frase_reflejada.append(cambio_pronombres.get(palabra,
palabra))
```





```
        return "Dices que " + " ".join(frase_reflejada) + "?"
    else:
        return random.choice(respuestas)
    return random.choice(respuestas_genericas)

# --- Interfaz de Streamlit ---

st.title("🗉 Chatbot Terapéutico ELIZA")
st.write("Hola, soy ELIZA. Estoy aquí para escucharte. Escribe 'adiós' para terminar.")

# Usamos st.session_state para mantener el historial de la conversación.
# Esto es como la "memoria" de la aplicación web.
if 'historial' not in st.session_state:
    st.session_state.historial = []

# Mostrar los mensajes anteriores
for mensaje in st.session_state.historial:
    with st.chat_message(mensaje["rol"]):
        st.markdown(mensaje["contenido"])

# Obtener la nueva entrada del usuario
if prompt := st.chat_input("Escribe tu mensaje aquí..."):
    # Mostrar el mensaje del usuario en la interfaz
    with st.chat_message("user"):
        st.markdown(prompt)
    # Añadir el mensaje del usuario al historial
    st.session_state.historial.append({"rol": "user", "contenido": prompt})

    # Generar la respuesta del chatbot
    respuesta_eliza = generar_respuesta(prompt)

    # Mostrar la respuesta del chatbot en la interfaz
    with st.chat_message("assistant"):
        st.markdown(respuesta_eliza)
    # Añadir la respuesta del chatbot al historial
    st.session_state.historial.append({"rol": "assistant", "contenido":
respuesta_eliza})
```

Explicación del Código 6:

Este código combina toda nuestra lógica anterior con las herramientas de Streamlit para crear la interfaz web.





1. **Lógica Reutilizada:** Primero, copiamos y pegamos toda la lógica de nuestro chatbot (los diccionarios `respuestas_por_palabra_clave`, `cambio_pronombres`, la lista `respuestas_genericas` y la función `generar_respuesta`) al principio del archivo.
2. **`st.title()` y `st.write()`:** Estos comandos de Streamlit se usan para mostrar el título y un texto de bienvenida en la página web.
3. **`st.session_state`:** Esta es una característica clave de Streamlit. Es un objeto similar a un diccionario que persiste mientras el usuario tiene la aplicación abierta. Lo usamos para crear y mantener `st.session_state.historial`, una lista que guarda cada mensaje de la conversación. Sin esto, la aplicación “olvidaría” los mensajes anteriores cada vez que el usuario envía uno nuevo.
4. **`st.chat_message()`:** Este comando crea los bonitos contenedores de chat para los mensajes, distinguiendo visualmente entre el “usuario” (user) y el “asistente” (assistant).
5. **`st.chat_input()`:** Este es el comando que crea el cuadro de texto en la parte inferior de la pantalla, permitiendo al usuario escribir y enviar sus mensajes. El `if prompt := ...` es una forma moderna de Python (llamada “expresión de morsa”) que asigna el texto del usuario a la variable `prompt` y comprueba si no está vacío, todo en una línea.

Paso 3: Ejecutar la Aplicación

Para lanzar la aplicación, abre una terminal, navega hasta el directorio donde guardaste `app.py` y ejecuta el siguiente comando.

```
# Código 7: Ejecución de la aplicación Streamlit
streamlit run app.py
```

Explicación del Código 7:

Este comando se ejecuta en tu terminal (no en la libreta de Jupyter).

1. Asegúrate de estar en el mismo directorio donde guardaste el archivo `app.py`.
2. Escribe `streamlit run app.py` y presiona Enter.

Streamlit iniciará un pequeño servidor web en tu computadora y abrirá una nueva pestaña en tu navegador con tu aplicación de chatbot lista para usar. ¡Ahora tienes una interfaz gráfica interactiva para tu ELIZA!





Conclusión

¡Felicidades! Has construido un chatbot funcional desde cero utilizando solo los fundamentos de Python. A lo largo de este proyecto, has aprendido de manera práctica sobre:

- **Estructuras de datos** como diccionarios y listas para organizar las reglas y respuestas.
- **Manipulación de cadenas (str)** con métodos como `.lower()`, `.split()` y el operador `in` para analizar la entrada del usuario.
- **Estructuras de control de flujo** como bucles `for` y `while` para iterar sobre datos y mantener la conversación.
- **Condicionales** (`if/elif/else`) para tomar decisiones lógicas.
- **Funciones** (`def`) para encapsular y reutilizar código.

Este enfoque demuestra que no siempre se necesitan herramientas complejas para construir proyectos interesantes. Con una buena lógica y un dominio de los fundamentos, puedes crear aplicaciones sorprendentes. Ahora tienes una base sólida para explorar proyectos más avanzados en el campo de la inteligencia artificial y el procesamiento de lenguaje natural.

